Predicting Novice Programmer Performance from AI-Tutor Interaction Logs Using Machine Learning

Anissa Williams
MS Data Science & Analytics
College of Charleston
April 2026

***Abstract*—This study investigates whether interaction-level behavioral signals from an AI tutoring system can predict student performance in programming tasks. The study was conducted with second-semester College of Charleston Computer Science students in Spring 2026. The tutoring system generates rich interaction logs that capture how students engage with instructional content moment-to-moment. This work uses those logs to investigate whether session-level behavioral signals, such as timing, pacing, and interaction volume, can predict student performance in a programming context. Using 104 complete learning sessions, we first construct a 16-feature model incorporating engagement, temporal, metadata, and difficulty-weighted features. These models achieve strong predictive performance (AUC-ROC up to 0.79), but also exhibit substantial overfitting, with Random Forest and Gradient Boosting reaching training accuracies of 0.952 and 1.00, respectively.**

Feature analysis reveals that the difficulty-weighted features introduce data leakage by partially encoding quiz outcomes. To address this, we refine the model to a leakage-free, five-feature behavioral set. On this more rigorous formulation, the strongest model (SVM with an RBF kernel) achieves 0.714 test accuracy, 0.681 cross-validated accuracy, and a 0.764 AUC-ROC, demonstrating that behavioral interaction traces alone can predict performance above chance.

Interpretability analysis using SHAP identifies session duration, response-time pacing, and interaction structure as the most influential predictors. These findings demonstrate the feasibility of real-time, behavior-driven intervention in AI tutoring systems and highlight the importance of rigorous feature design in educational machine learning.

1. Introduction

Artificial Intelligence in Education (AIED) has increasingly focused on adaptive systems that personalize instruction and provide timely feedback. A central goal in this domain is the early identification of students at risk of poor learning outcomes, enabling intervention *during* the learning process rather than *after* assessment. Predictive modeling plays a key role in this shift from reactive to proactive education.

Recent advances in large language models (LLMs) have introduced conversational AI tutoring systems that interact with students in natural language. These systems can generate fine-grained behavioral traces, including timing, pacing, and message exchange patterns, providing a continuous record of student engagement. Prior work in educational data mining (EDM) shows that behavioral patterns predict learning outcomes even when no answer-level information is available [1].

This study investigates whether interaction-level behavioral signals from an AI tutor can predict student performance in a programming context. Using data from a controlled Java tutoring experiment, we analyze session-level interaction logs to evaluate the predictive power of behavioral features.

A key contribution of this work is the identification and correction of a subtle but critical form of data leakage: difficulty-weighted features derived from quiz responses can partially encode the target variable, artificially inflating model performance. By removing these features, we demonstrate that a compact set of behavioral signals alone retains meaningful predictive power.

Unlike many prior studies that rely on observational or platform-aggregated datasets, this work leverages a controlled dataset collected within a university programming course, capturing complete conversational interaction logs between students and an AI tutor. This setting enables detailed analysis of behavioral dynamics during learning, providing a level of interaction detail that is not typically available in standard educational datasets.

Specifically, this paper makes four contributions:

- Demonstrates that AI tutor interaction logs contain predictive behavioral signals
- Identifies and corrects data leakage introduced by quiz-derived features
- Shows that behavioral-only models achieve robust performance without reliance on correctness-based inputs
- Evaluates these methods on a controlled, conversational dataset collected from a real-world AI tutoring deployment

2. Dataset Description

A. Source Data

The dataset consists of 104 completed learning sessions from 52 students enrolled in a second-semester Java programming course. Students interacted with an AI Java tutor across two topics: ArrayLists and Recursion. Each session includes full conversational logs, timestamps, session metadata, and quiz results.

The raw dataset contains approximately 28,600 lines of JSON logs, which were parsed into structured tables representing messages, sessions, and quiz responses. Because the analysis examines behavioral patterns across repeated sessions for each student, only learners who completed both topics were retained.

B. Target Variable

Student performance is defined as a binary classification based on quiz scores:

- High performance: $\geq 70\%$
- Low performance: $< 70\%$

The resulting distribution is approximately balanced, with 49 High (47.1%) and 55 Low (52.9%), supporting reliable model evaluation.

The mean quiz score across all conditions is 66.9% and the median is 60%, reflecting a slight left skew. This distribution was used for all stratified splits and cross-validation folds.

C. Feature Engineering

Features were engineered across four categories:

- Engagement: total messages, user messages, assistant messages
- Temporal: session duration, average response time, response-time variability
- Metadata: instructional condition, session type
- Difficulty-weighted (later removed): correctness weighted by question difficulty

This produced an initial set of 16 features used for exploratory modeling.

TABLE 1: Full Feature List

Feature	Category
total_messages	Engagement
condition	Metadata
session_type_enc	Metadata
has_both_enc	Metadata
duration_seconds	Temporal
avg_response_time	Temporal
min_response_time	Temporal
std_response_time	Temporal
median_response_time	Temporal
assistant_messages	Engagement
user_messages	Engagement
rapid_response_pct	Temporal + Engagement
rapid_response_count	Temporal + Engagement

3. Preprocessing

The raw dataset consisted of 28,600 lines of JSON logs generated by the AI Java Tutor. Each JSON record contained nested structures for messages, timestamps, quiz responses, and metadata. To prepare these logs for modeling, we implemented a multi-stage preprocessing pipeline that transformed the raw JSON into structured, analysis-ready tables.

A. JSON Parsing and Flattening

The first stage involved converting the nested JSON logs into relational tables. Three primary tables were produced:

- **messages.csv**: one row per message, containing sender (user or assistant), timestamp, and message text.
- **sessions.csv**: one row per session, containing session-level metadata such as topic, instructional condition, and duration.
- **quiz.csv**: one row per quiz item, including correctness and difficulty level.

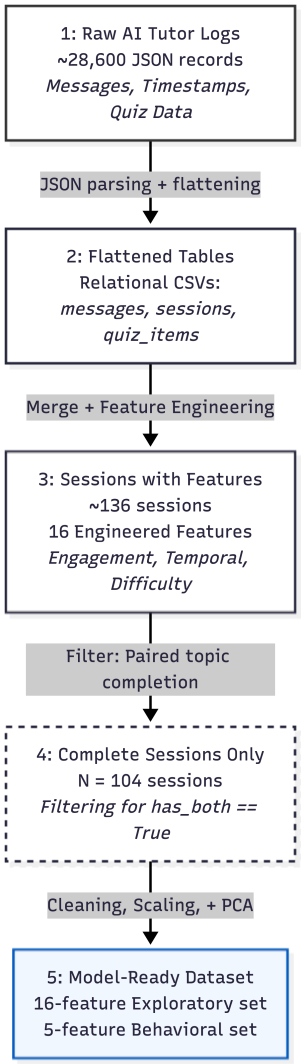

Fig. 1: Preprocessing Pipeline

Flattening the logs enabled feature engineering at both the message level (e.g., response times) and the session level (e.g., total messages, duration).

B. Session Merging and Feature Assembly

The flattened tables were merged into a unified dataset, **sessions_with_engagement_features.csv**, which contained all engineered features required for modeling. This merge included:

- computing response-time features by differentiating consecutive message timestamps,
- aggregating message counts (total, user, assistant),
- calculating rapid-response indicators,
- attaching quiz-level difficulty information,
- creating a boolean feature where the user id for both sessions matched and the sessions were completed

C. Filtering for Complete Sessions

A critical pre-processing step was filtering to sessions where students completed *both* topics (ArrayList and Recursion).

This was implemented using the engineered boolean field `has_both`.

This step reduced the dataset from 136 to 104 complete sessions, giving each student a matched pair of sessions and preventing the analysis from being skewed by topic-specific differences.

D. Cleaning and Imputation

Several fields required cleaning:

- missing difficulty values were imputed with 0,
- response-time nulls were filled with the median response time for that session,
- boolean fields were converted to integers,
- categorical fields (e.g., session type, instructional condition) were label-encoded.

These steps ensured that all features were numeric and compatible with downstream models.

E. Scaling

All numeric features were standardized using `StandardScaler`, producing zero-mean, unit-variance inputs. Scaling was particularly important for SVM, which is sensitive to feature magnitude.

F. Leakage Identification and Feature Refinement

During feature importance analysis (Fig 2), we identified a critical issue: difficulty-weighted features were derived from quiz responses and therefore partially encoded the target variable. This constitutes data leakage, leading to inflated or misguided performance estimates.

To address this, we removed all quiz-derived features. Additionally, the five response-time variants (avg, median, std, min, max) capture highly correlated information, with avg_response_time providing the strongest and most interpretable signal. min_response_time was the original pacing feature but reflects only a single moment rather than sustained behavioral patterns. It was replaced later by avg_response_time in the final pipeline.

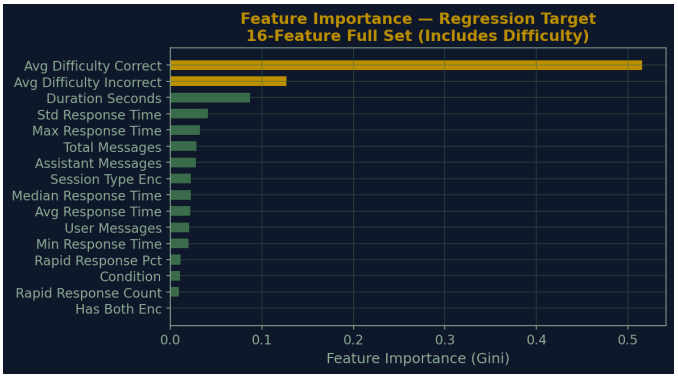

Fig. 2: Feature Importance for the 16-feature classification model (includes difficulty-weighted features).

To address the leakage and correlation issues, we refined the dataset to a five-feature behavioral-only model containing no quiz-derived information:

- condition,
- session_type_enc,
- duration_seconds,
- total_messages,
- avg_response_time.

These results show that even when the target-based (quiz score) features are removed, the behavioral traces from a session still predict learning outcomes at levels above chance.

4. Machine Learning Models

We evaluate four classifiers:

- Logistic Regression
- Gradient Boosting (100 estimators, max_depth=3)
- Random Forest (100 trees, max_depth=5)
- Support Vector Machine (RBF Kernel)

Models are trained using an 80/20 stratified split and evaluated with 5-fold cross-validation. Performance is reported using train accuracy, test accuracy, F1-score, AUC-ROC, and 5-fold cross-validated accuracy.

5. Results

TABLE 2: Full (16) Feature Model Performance Results

Model	Train	Test	F1	AUC	CV
Logistic Regression	0.675	0.571	0.566	0.591	0.591
Random Forest	0.952	0.667	0.667	0.673	0.577
Gradient Boosting	1.000	0.667	0.667	0.791	0.624
SVM (RBF)	0.735	0.571	0.566	0.618	0.681

A. Full Feature Model Performance (16-Feature Set)

Table 2 summarizes performance for the full 16-feature model, which includes the difficulty-weighted features. The performance appears high, but the difficulty-weighted features embed target information that inflates both training and test results.

Across models, test accuracy ranges from 0.57 to 0.67. Gradient Boosting achieves the highest AUC-ROC (0.791) and reaches perfect training accuracy (1.00). The gap between train and test accuracy indicates overfitting driven by the leaked difficulty features. Random Forest shows a similar pattern, with high training accuracy (0.952) and substantially lower cross-validated accuracy (0.577).

SVM with an RBF kernel demonstrates the most stable generalization, with consistent five-fold CV accuracy (0.681). This suggests that the kernel model is capturing meaningful behavioral structure rather than relying on leaked difficulty signals. These patterns motivated the shift to a leakage-free behavioral feature set.

B. Behavioral Model Performance (5-Feature Behavioral Set)

After removing difficulty-weighted features, model performance reflects purely behavioral prediction. Table 3 shows that SVM (RBF) achieves the strongest overall performance on the clean behavioral-only feature set, with a test accuracy of 0.714, cross-validated accuracy of 0.681, and an AUC-ROC of 0.764.

TABLE 3: Behavioral-Only Feature (5) Model Performance Results

Model	Train	Test	F1	AUC	CV
Logistic Regression	0.663	0.667	0.667	0.755	0.642
Random Forest	0.964	0.619	0.617	0.691	0.681
Gradient Boosting	1.000	0.667	0.651	0.809	0.557
SVM (RBF)	0.735	0.714	0.714	0.764	0.681

SVM with an RBF Kernel provides the strongest and most stable performance on the behavioral-only feature set, which suggests that nonlinear decision boundaries fit the structure of the interaction-log data. Logistic Regression also performs competitively (0.667 test accuracy), indicating that even a linear boundary captures meaningful behavioral signal. Random Forest shows improved stability relative to the full feature set, with cross-validated accuracy increasing to 0.681. This improvement reflects the reduction in overfitting once the leakage-prone difficulty features are removed.

C. Confusion Matrix Analysis

Below we analyze the confusion matrices for the 5-feature behavioral set across the two strongest models: Logistic Regression (best linear baseline) and SVM (RBF) (best overall classifier).

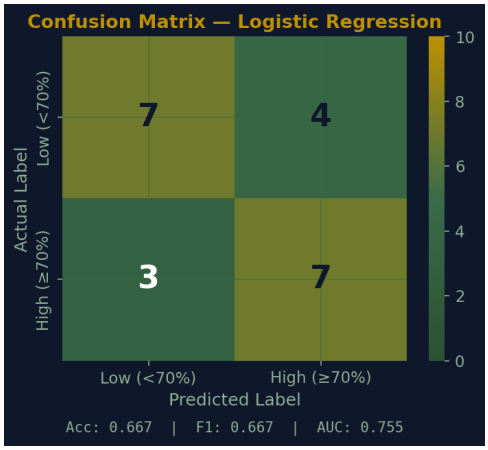

Fig. 3: Confusion matrix for Logistic Regression

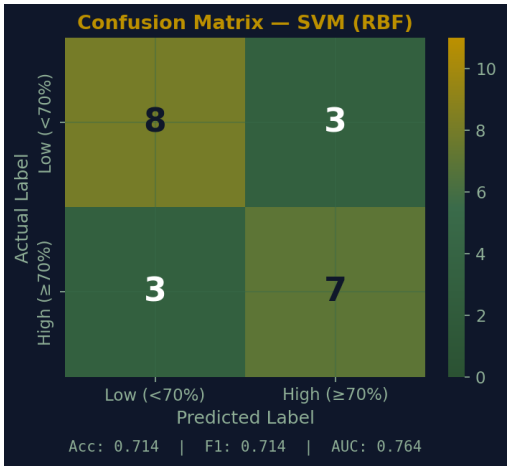

Fig. 4: Confusion matrix for SVM (RBF)

The confusion matrices highlight distinct differences in model behavior:

- **Logistic Regression** provides a strong linear baseline, capturing much of the behavioral signal with relatively balanced performance across classes.
- **SVM (RBF)** provides the most stable and accurate classification, with strong performance in both classes and fewer extreme errors.

Errors cluster around students near the performance threshold (60–75%), which suggests that borderline learners show less distinct behavioral patterns. Taken together, these results show that behavioral features are predictive of student performance, but the signal becomes less reliable for learners close to the mastery cutoff.

D. Feature Importance and SHAP Interpretability

Interpretability analyses were conducted using both model-based (Random Forest Gini importance) and model-agnostic (SHAP) methods. On the full 16-feature set, difficulty-weighted features dominate importance rankings, confirming leakage. However, timing and pacing features still contribute meaningfully.

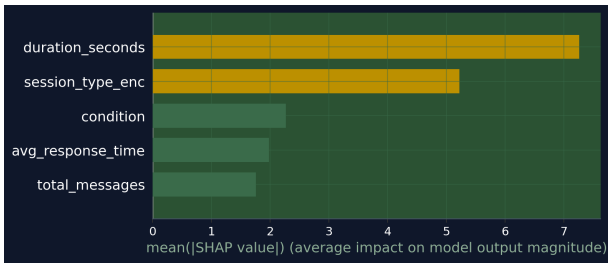

Fig. 5: SHAP feature importance for the 5-feature model.

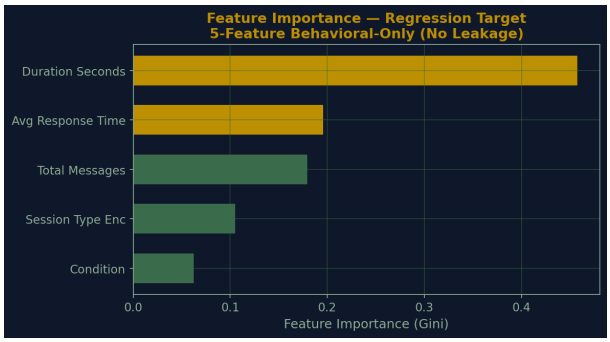

Fig. 6: Gini feature importance for the 5-feature model.

On the refined 5-feature behavioral model, SHAP reveals a consistent global pattern:

- **duration_seconds** is the strongest predictor
- **session_type_enc** and condition provide structural context
- **avg_response_time** captures pacing and cognitive load
- **total_messages** reflects engagement depth

The agreement between Gini importance and SHAP strengthens confidence that these behavioral signals are robust and not artifacts of a single model.

6. Related Work

Research in Artificial Intelligence in Education (AIED) spans intelligent tutoring systems, adaptive feedback, and student modeling [2]. Early intelligent tutoring systems (ITS) relied on rule-based and model-tracing architectures to provide structured scaffolding and individualized support, demonstrating consistent learning gains in domains such as programming [3]. Machine learning has long played a role in these systems: tutors use ML techniques to infer student knowledge, identify learning strategies, and adapt instruction based on patterns in tutor-student interactions [2]. These approaches show that behavioral traces, rather than answer-level information, can serve as meaningful signals for personalization.

Educational Data Mining (EDM) work has shown that behavioral features such as timing, pacing, hesitation, and engagement depth can predict student performance[1]. Large-scale studies of internet usage and LMS clickstream data demonstrate that interaction behaviors can differentiate performance groups and support early identification of at-risk learners [4]. These works highlight that temporal patterns, connection frequency, and engagement discipline are predictive of academic outcomes, even without access to correctness-based features. Recent MOOC research similarly shows that event-level interaction logs can be used to cluster learners and predict behavioral trajectories with high accuracy [5]. However, these systems depend on platform-level activity data rather than session-level learning interactions.

Clickstream-based predictive modeling has also been used in early warning systems. Borna et al. [9] showed that models trained on LMS interaction logs can identify at-risk students, although predicting high achievers remains difficult because engagement patterns do not map cleanly onto performance. This limitation highlights the need for more nuanced behavioral modeling approaches that move beyond coarse interaction counts. In programming education, prior work has focused on IDE activity data, code snapshots, and compilation events as predictors of learning outcomes [6], [7]. Much less attention has been given to conversational interaction logs from AI tutors, particularly those powered by large language models. Existing studies on LLM-based tutoring primarily examine learning gains, usability, or student perceptions [8], rather than treating the interaction itself as a behavioral dataset for predictive modeling. This work contributes to the literature by showing that session-level behavioral features extracted from LLM tutor logs can predict short-term quiz performance above chance, even without answer-level features. It also identifies and corrects a form of data leakage introduced by difficulty-weighted quiz features, underscoring the importance of careful feature design when modeling learning outcomes. To our knowledge, this is one of the first empirical analyses to use LLM tutor interaction logs from a controlled study as a predictive modeling dataset in novice programming contexts.

7. Discussion

A. Uniqueness and Ethical Considerations

Many predictive models in education rely on demographic variables, prior academic history, or socio-economic indicators. These features can improve accuracy, but they also risk reinforcing existing inequities by carrying structural biases into the model. Behavioral-only approaches offer a more equitable alternative because they focus on how students interact with the learning environment rather than who they are or what background they come from. By centering pacing, duration, and message patterns, this study avoids background-based features and instead models the learning process itself. The ablation results show that these behavioral features can predict performance above chance even when outcome-adjacent features are removed. This direction aligns with current ethical guidance in AIED that emphasizes fairness, transparency, and careful attention to bias.

This study treats AI tutoring interactions as a real-time behavioral dataset for predictive modeling. Unlike approaches that rely on post-hoc performance metrics, interaction logs provide continuous signals that can support earlier and more responsive intervention.

The results show that behavioral features such as pacing, duration, and interaction structure are sufficient to predict short-term performance above chance. These features are also actionable in practice: an AI tutor can detect rapid responses or shallow engagement and adjust its scaffolding in the moment.

Finally, the identification of leakage highlights the importance of careful feature design. Without proper validation, models can appear highly accurate while relying on signals that do not reflect genuine learning behavior.

B. Practical Implications for Adaptive Learning

This study demonstrates that interaction-log features provide a non-intrusive, real-time mechanism for diagnostic assessment. By prioritizing behavioral signals over performance-based metrics, tutoring systems can mitigate the risks of data leakage and provide pedagogical interventions, such as adjusting scaffolding depth or triggering metacognitive prompts, well before a student reaches the final assessment phase. These results position the AI tutor not as a static instructional tool, but as a sensitive diagnostic engine capable of flagging behavioral patterns associated with cognitive struggle.

8. Conclusion

A. The Impact of Feature Refinement and Leakage Removal

Initial modeling on the full 16-feature set yielded performance metrics that, while seemingly strong, were identified as being artificially inflated by data leakage. Feature importance analysis revealed that difficulty-weighted metrics, derived from quiz responses, partially encoded the target variable. This realization necessitated a pivot to a more rigorous analytical narrative focused exclusively on session-level behavioral signals.

B. Interpretability and Behavioral Signals

The convergence of SHAP values and Gini importance confirms that session duration, message pacing (average response time), and engagement depth (total messages) are the most robust indicators of short-term mastery. On the leakage-free behavioral formulation, the Support Vector Machine with an RBF kernel achieved the strongest and most stable performance (test accuracy = **0.714**, 5-fold CV = **0.681**, AUC-ROC = **0.764**), indicating the strength of behavioral signals and demonstrating that AI tutoring systems can identify at-risk students in real time based on conversational cadence.

REFERENCES

[1] I. Papadogiannis, M. Wallace, and G. Karountzou, "Educational data mining: A foundational overview," *Encyclopedia*, vol. 4, no. 4, pp. 1644–1664, 2024. [Online]. Available: https://www.mdpi.com/2673-8392/4/4/108

[2] B. P. Woolf, "AI in education," *AI Magazine*, vol. 34, no. 4, pp. 80–93, 2013.

[3] K. R. Koedinger, J. Kim, J. Jia, E. McLaughlin, and N. Bier, "Learning is not a spectator sport: Doing is better than watching for learning from a MOOC," pp. 111–120, 2015.

[4] X. Xu, J. Wang, H. Peng, and R. Wu, "Prediction of academic performance associated with internet usage behaviors using machine learning algorithms," *Computers in Human Behavior*, vol. 98, pp. 166–173, 2019.

[5] J. Chonraksuk and S. Boonlue, "Development of an AI predictive model to categorize and predict online learning behaviors of students in Thailand," *Heliyon*, vol. 10, no. e32591, 2024.

[6] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," in *Advances in Neural Information Processing Systems*, 2015, pp. 505–513.

[7] K. Rivers and K. R. Koedinger, "Automated data-driven feedback for novice programmers," in *Proceedings of the 2017 ACM Conference on Learning at Scale*, 2017, pp. 1–10.

[8] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, M. Göcks, and G. Kasneci, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.

[9] M. Borna, J. Siraj, and R. Ghanbarzadeh, "Predicting student performance from LMS interaction logs using machine learning methods," *Frontiers in Education*, vol. 9, 2024.